ARCHITECTURE NOTE · 04

Glassbox agents

The architecture proposal, the reference implementation that proves it runs, and the build I actually use to ship agents today.

The three tools in the other notes are each one service line's answer. This one is the general shape underneath them, and it started as a written blueprint before any of them existed: a **two-lane agentic platform** — a sovereign local lane for confidential client work, a cloud lane for everything else, sharing one data layer, one orchestration standard and one governance plane.

The argument for that shape is not technical. Under the EU AI Act, NIS2 and DORA, the defensible differentiator for a professional-services firm is not agent capability; it is being able to show a regulator, a client and your own quality function exactly what an agent saw, decided and did. Governance is the product, not the overhead.

glassbox-agents is that blueprint as working, tested Python — zero third-party dependencies in the core, Apache-2.0 — so the claims can be run rather than read.

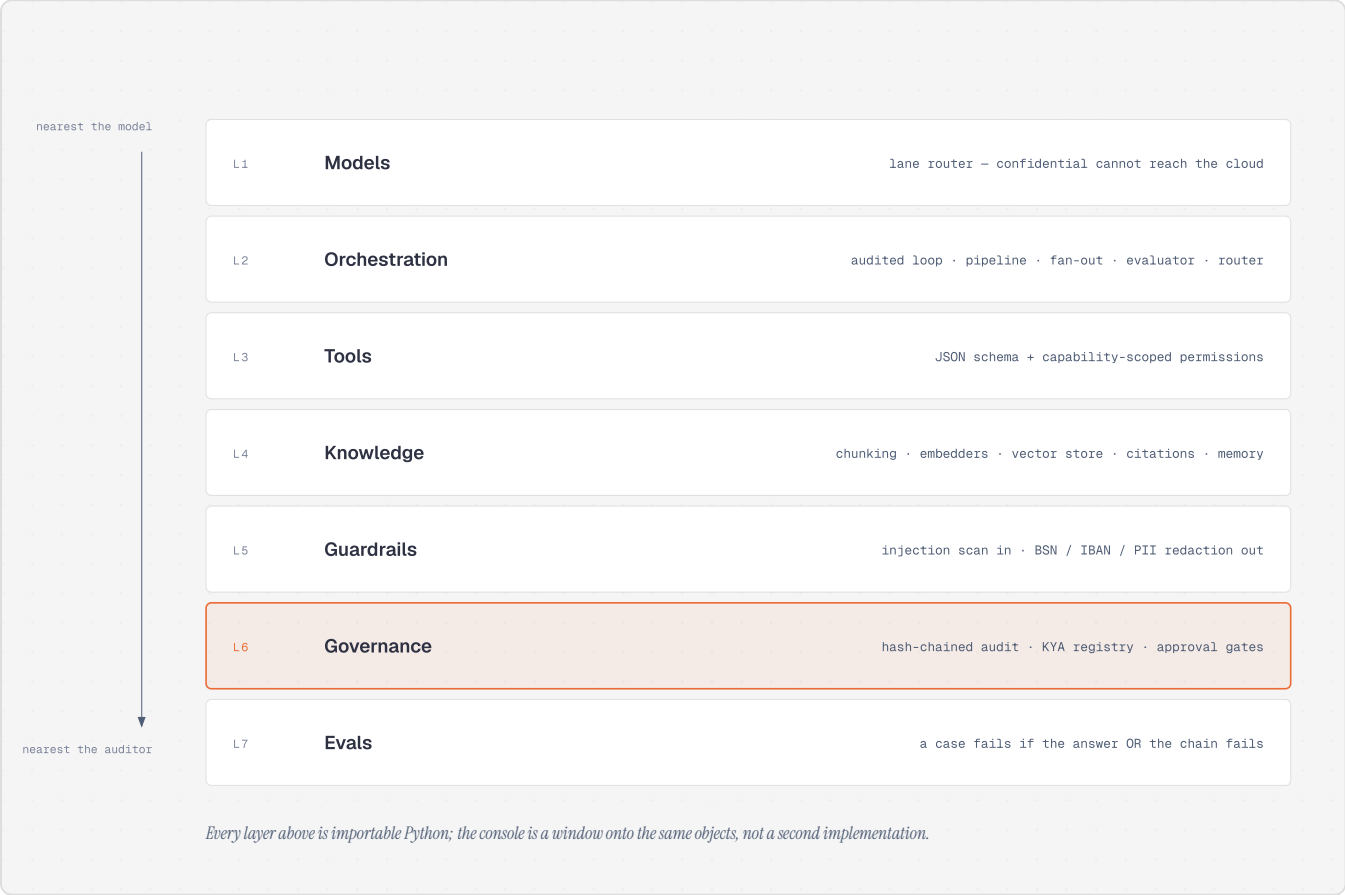

Fig. 1 — the seven layers. Each is importable on its own; the web console is a window onto the same objects, not a second implementation.

The one control that is structural

Most "the data stays in the EU" claims are policy documents. This one is a type error. A task carries a sensitivity label; a confidential task pinned to the cloud lane raises a `LaneViolation` rather than falling back. You can demonstrate it in the console in ten seconds by pinning the lane on a confidential engagement and pressing run.

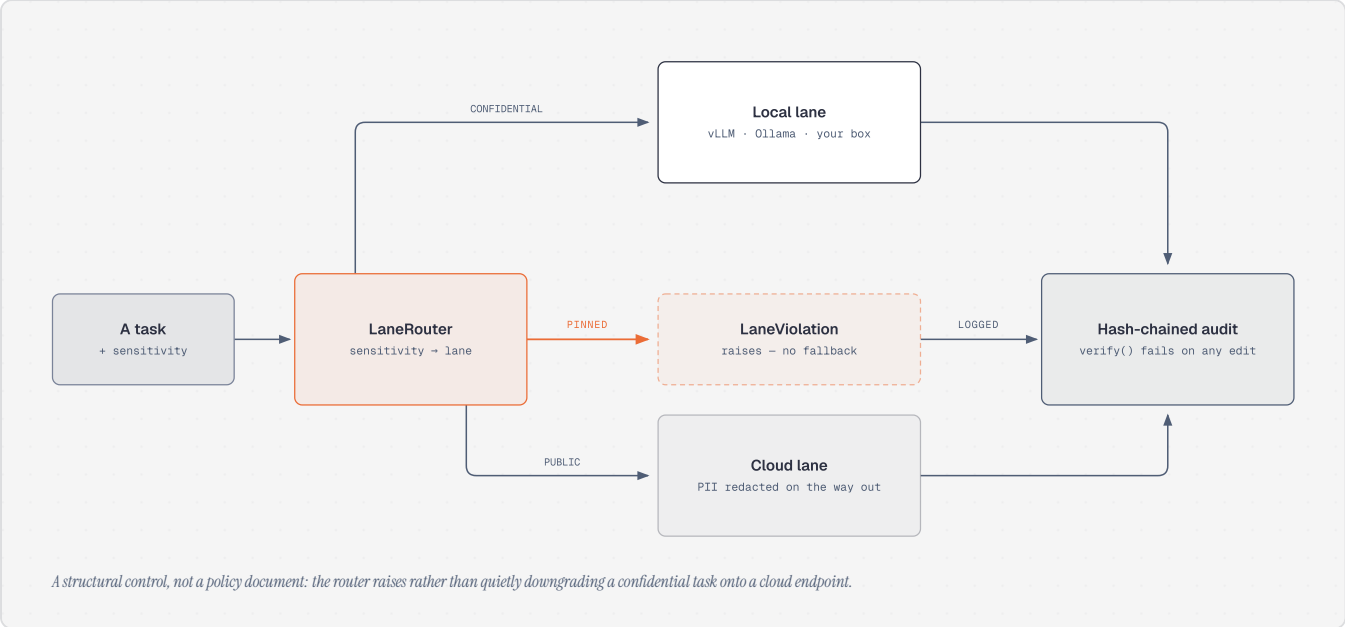

Fig. 2 — the router raises; it does not downgrade.

The rest of the governance plane works the same way. Every step appends to a SHA-256 hash chain, so editing, deleting or reordering a past record makes `verify()` fail — and the exported JSONL can be verified independently later. High-risk actions stop and wait for a human. Agents act under a registered identity with a scoped capability set and instant revocation: revoke one and the next run is denied before the model is called. A prompt-injected agent still cannot call a tool outside its capability set.

Where I stand on guardrails

I do not treat prompt engineering as a guardrail. A prompt is the right place to describe the behaviour I want and it is the layer a model can be argued out of — by a user, by a document it retrieved, or by its own drift. It cannot enforce a permission, prevent a tool call, prove what happened afterwards, or refuse on my behalf.

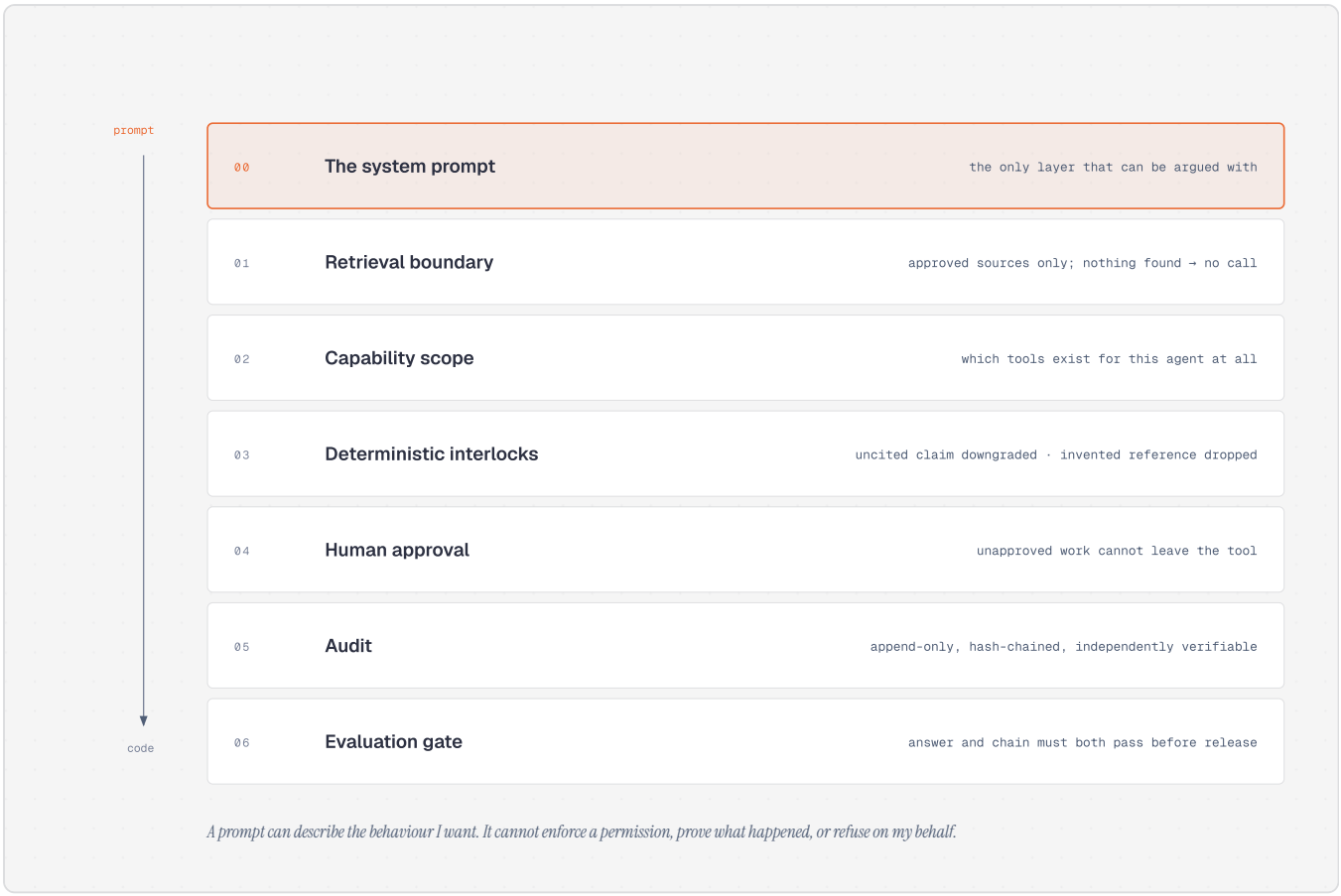

Fig. 3 — the layers that actually hold, and the one that does not.

Every interlock in the other three notes is an instance of this: the destructive-command regex that overrides a standing grant, the empty retrieval that skips the second model call, the citation outside the supplied range that is discarded, the past client name that refuses approval rather than warning about it. None of them are instructions to the model. Each one is a few lines of code written after watching a model do the thing it prevents.

> *The test I apply: if the model ignored every word of its system prompt, what would still stop it? Whatever survives that question is the guardrail. The rest is a preference.*

How I actually build and ship an agent

The delivery side is deliberately boring, and it is shaped by one real constraint: the Mac I author on has no tenant access, and the laptop with tenant access is not where I want an agent writing code. So authoring and deployment are physically separated, and GitHub is the only thing that crosses.

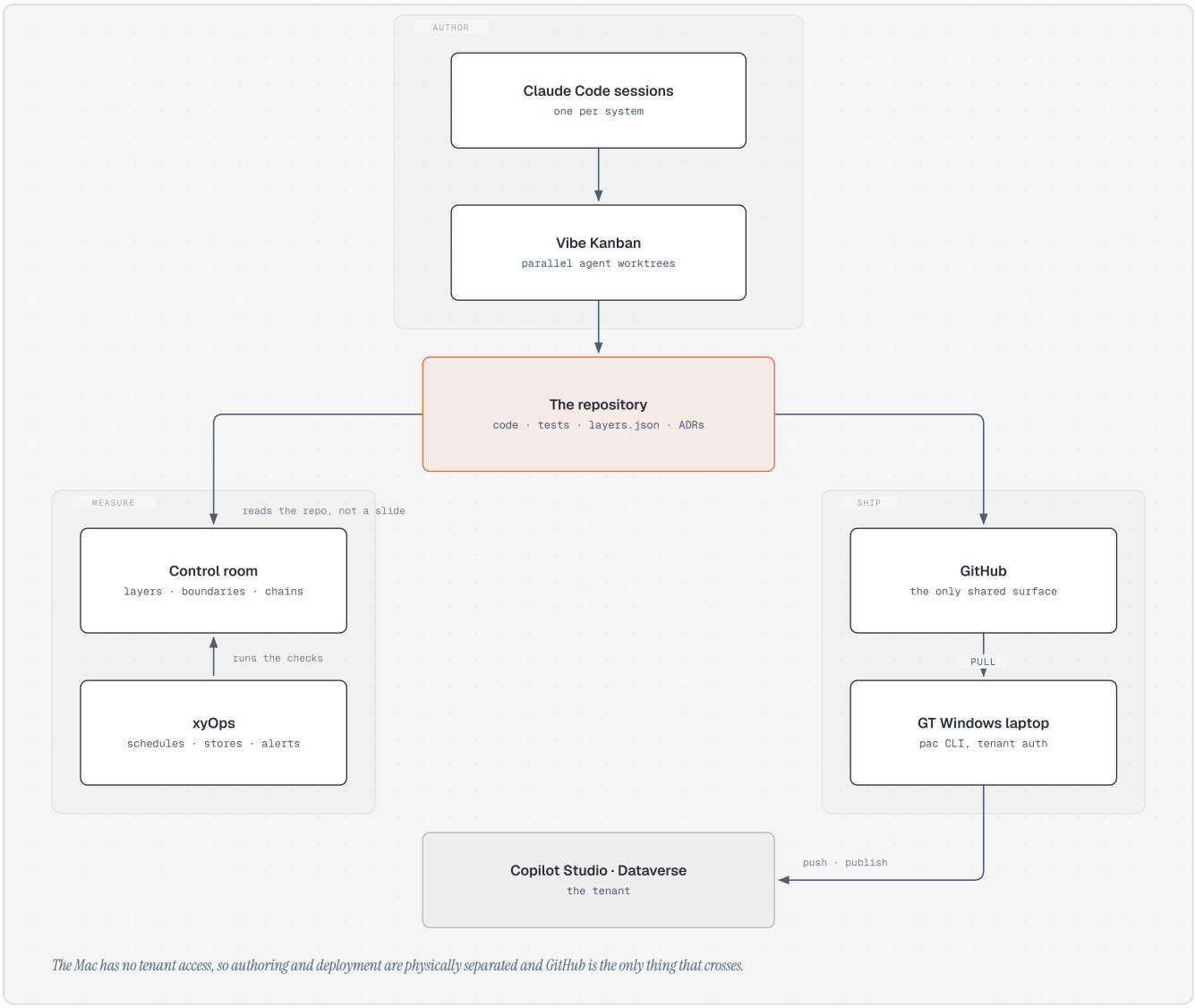

Fig. 4 — prompt to tenant. Two machines, one shared surface.

STAGE	WHAT IT IS
Author	Claude Code sessions, one per system, running in parallel Vibe Kanban worktrees so several agents can work without sharing a working tree.
The repo	Code, tests, ADRs, and a `layers.json` that declares the system's boundary — what it reads, with which permission, where the data sits, what leaves. A new system declares its boundary before it has code.
Measure	A control room reads that file *and the code*: which layers are built and green, which boundaries the code actually respects, every test with its last line, the hash chains. xyOps schedules those checks, stores the results and alerts.
Ship	GitHub carries the YAML to the Windows laptop, where `pac` authenticates against the tenant and pushes into Copilot Studio and Dataverse.

The part I would defend hardest is that the control room measures the repository rather than a status slide. A boundary claim that no code satisfies shows up red,

and so does a test whose last line is a failure. It is the same principle as the interlocks, applied one level up.

The worked example

The current agent on that pipeline is a forensics agent for GT: it answers questions about published Dutch fraud rulings from a curated corpus of 561 judgments, extracting court, date, ECLI, alleged offences and forensic methods with citations — and, deliberately, answers nothing else. It is locked down in YAML: `useModelKnowledge: false`, `webBrowsing: false`. Anything not in the corpus gets *"Dat staat niet in de verzamelde uitspraken."* For an audience of forensic accountants, an agent that refuses is worth more than one that improvises.

The same corpus also runs as an MCP server with three tools, outside Microsoft entirely — no tenant, no licence, no SharePoint — so the identical agent works in Claude Code, Claude Desktop or Cursor. That side-by-side is the point: one corpus, two agents, one of which needs a tenant and a licence and one of which needs neither.

What I would want a reviewer to push on

- The layers behind the production seams — vector store, embedder, chat model, approval gate — are in-memory reference implementations. The seams are real; what is behind them is not production.
- Console engagements do not survive a restart. The audit trail does. That asymmetry is deliberate and I am not certain it is right.
- The guardrails layer ships pattern-based injection and PII detection. It is a floor, and I have no adversarial evaluation of where it stops working.
- The measurement plane checks that a boundary is respected by the code in the repo. It cannot check the boundary was the right one.
